

Stacks

A stack is a data structure that stores elements using Last-In-First-Out (LIFO) order: the last element added is the first to be removed.

Natural Explanation

Think of a stack of plates: you place plates on top, and you remove from the top. The last plate you add is the first one you take off.

Core Operations

PUSH(x): Add element **x** to the top of the stack.

POP(): Remove and return the element at the top.

TOP() / **PEEK()**: Return the top element without removing it.

ISEMPTY(): Check if the stack contains no elements.

All operations run in $O(1)$ time.

Typical Implementation

Stacks are easily implemented using a dynamic array (or linked list):

With a dynamic array: - Maintain an index **top** starting at -1 or 0 - **PUSH** appends to the array and increments **top** - **POP** returns the element at **top** and decrements

With a linked list: - Push/pop always work at the head - No need to resize or rebalance

Common Uses

- **Function call stack:** managing nested calls and return addresses
 - **Expression evaluation:** handling parentheses and operator precedence
 - **Undo/redo:** storing previous states
 - **Depth-first search:** exploring paths recursively
 - **Backtracking problems:** exploring alternatives and undoing choices
-

LIFO Property

The distinguishing property is that access is restricted to the top. You cannot directly access or modify the middle of a stack without removing elements first.

Related

- [\[\[Queues\]\]](#)
- [\[\[Linked Lists\]\]](#)
- [\[\[Depth-First Search\]\]](#)
- [\[\[Divide and Conquer\]\]](#)